DEVELOPING CODE RECIPES FOR AN EARTH SCIENCE DATA ANALYSIS LIBRARY

By Natalia Hunt

The use of programming in Earth Science for Data analysis has become increasingly important to produce plots, analyse data and predict future patterns. Tools such as cf-python and cf-plot have made these tasks more accessible to scientists. During my placement, I developed a set of recipes to facilitate data visualisation in the netCDF format. These recipes include: 1) generating subplots with various map projections, 2) creating subplots with different colour maps/scales to minimise bias in data presentation, and 3) calculating correlations between datasets, specifically examining the positive correlation between snow coverage and land elevation across a region in England, supplemented with profile and contour plots.

Abstract 2
Introduction 2
Methodology 3
Installing packages and programming environments 3
First Recipe – Subplots with different projections 3
Second Recipe – Subplots with different Colour Maps 5
Third Recipe – Comparing two datasets: Snow against elevation 6
Results and Discussion 8
First Recipe – Subplots with different projections 8
Second Recipe – Subplots with different Colour Maps 10
Third Recipe – Comparing two plots: Snow against elevation 13
Conclusion 16
Appendix 17
References 21
Bibliography 24
Acknowledgements 24

Abstract

The use of programming in Earth Science for Data analysis has become increasingly important to produce plots, analyse data and predict future patterns. Tools such as cf-python and cf-plot have made these tasks more accessible to scientists. During my placement, I developed a set of recipes to facilitate data visualisation in the netCDF format. These recipes include: 1) generating subplots with various map projections, 2) creating subplots with different colour maps/scales to minimise bias in data presentation, and 3) calculating correlations between datasets, specifically examining the positive correlation between snow coverage and land elevation across a region in England, supplemented with profile and contour plots.

Introduction

Earth Science requires vast amounts of data to be analysed and interpreted, NCAS CMS ("NCAS CMS," n.d.) has created their own packages, cf-python and cf-plot (Hassell & Bartholomew, 2020), making use of existing libraries and languages such as matplotlib and python. This allows for this data to be quickly analysed and graphs to be produced. This abstraction allows scientists to use a more accessible programming interface to interpret their data.

To begin I learnt what a code "recipe" is, ("Recipes using cf — Documentation," n.d.), on matplotlib they call these examples ("Examples — Matplotlib 3.9.1 documentation," n.d.). Code recipes are pre-written code that can be implemented by the user for a specific purpose, they can then edit it to use their own data, make change to it or use it alongside their own code, it is also useful for explaining topics and methods that cannot be explained using one function. This was implemented with NetCDF which is a data format making use of CF (Climate and Forecast) data conventions (Hassell et al., 2017), which is a guide for how metadata should be formatted in science, such as through standard names of different values ("CF Standard Names," n.d.). This ensures that when data sets are used computers can "understand" what they mean as the information is encoded in the same way.

Throughout my placement, my goal was to add to the existing set of recipes available for Earth Science Data Analysis, particularly making it clear what methods are available to represent data. (*A Brief Introduction to CF*, 2022) ("CF Conventions," n.d.)

Methodology

Installing packages and programming environments

Firstly, I needed to ensure I had the correct coding environment, this involves a lot of setup. Using a Linux (Mint) system ("Download LMDE 6 - Linux Mint," n.d.) meant that it was easier to work with data and code files via the terminal as well as make use of the python interactive mode, so I could quickly save plots. I installed python 3 ("Download Python," n.d.) to enable this and then anaconda, also known as conda, a package provider and environment manager ("Installing conda — conda 24.7.2.dev28 documentation," n.d.). Finally, I ensure the as cf-python, cf-plot and netCDF4-python packages ("NCAS-CMS/cf-python," September 13, 2019/2024) ("Download/Install — cf-plot 3.3.0 documentation," n.d.) ("Unidata/netcdf4-python," January 13, 2014/2024) were installed to allow me to start working on plots and being used as dependencies to add functionality.

At the start I used Jupyter Notebooks ("Project Jupyter," n.d.) for writing and commenting on my code, but as I had issues with esmpy not installing and causing issues on my main environment, I instead used VS Code. This meant meant I could have a separate environment open (with the esmpy package installed) with my netCDF data files accessible and text files where I could run my python script. Therefore, I could test my code quickly and have my plots saved in the same folder, ready to view and decide what change need to be made to the code.

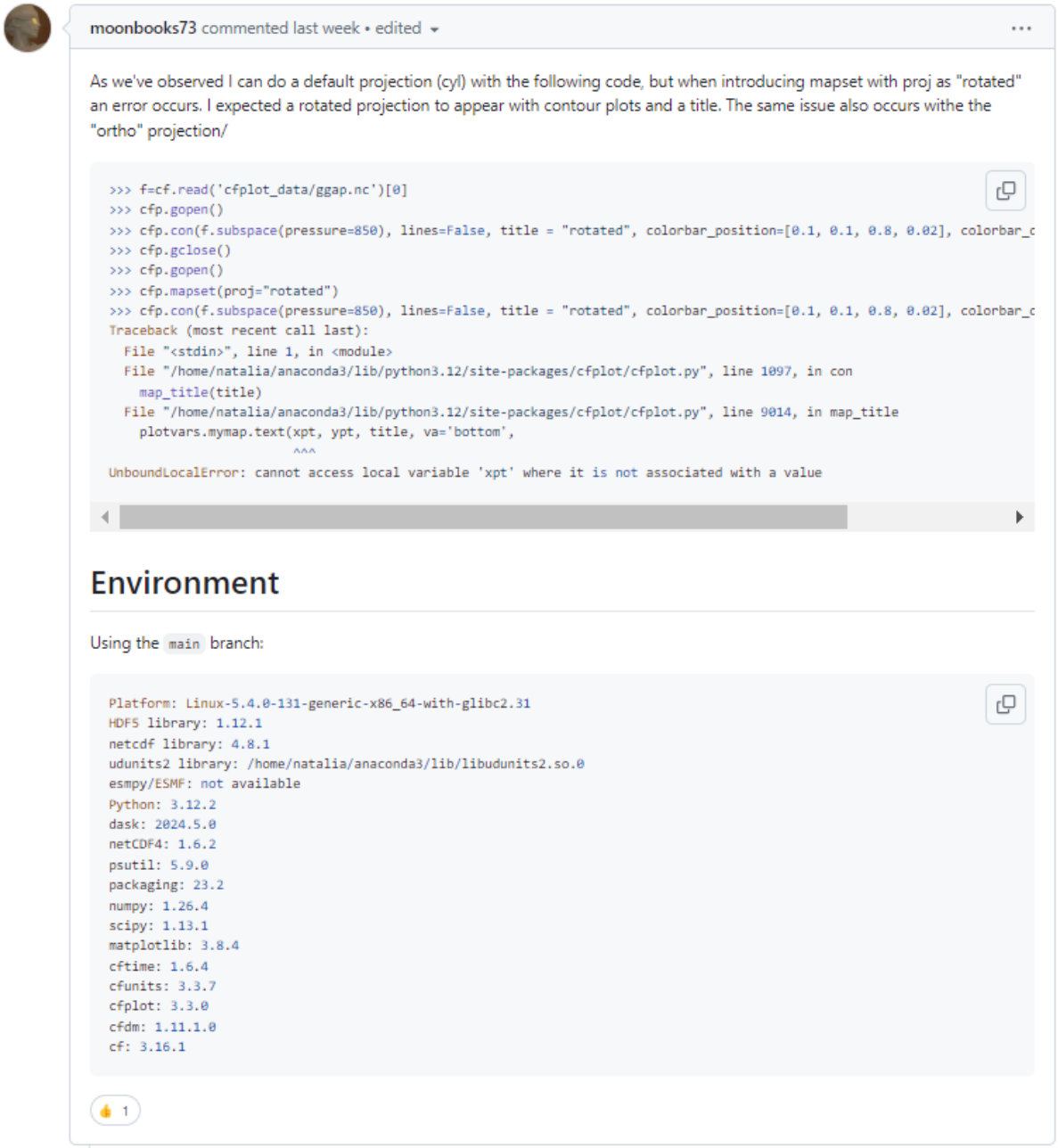
Following on from this, I used the cf-tools-training data to begin to understand how cf-python and cf-plot work together ("NCAS-CMS/cf-tools-training: NCAS data tools training material," n.d.) as well as videos on CF conventions ("CF Conventions," n.d.). Throughout my projects I use the data provided, such as the cfplot_data available from the sample data sets ("Download/Install — cf-plot 3.3.0 documentation," n.d.). I also used other data sets such as ("data: NOAA NGDC GLOBE topo," n.d.). This is because it is more helpful to demonstrate use of real-life data for recipes, so they could be more easily adapted to user's needs. This data is also higher resolution and although had some metadata issues was easily adjustable. Finally, I installed micromamba in order to use the esmpy library which is required for regridding ("Micromamba Installation — documentation," n.d.)

First Recipe – Subplots with different projections

I then started on my first recipe, one to show what all the various data projections (different ways of showing the data on different map forms) from cf-plot look like. There are a few different projections, the default is "cyl" and they all have shortened names used when programming such as "npstere", "spstere", "moll" and lcc ("Projections in cf-plot — cf-plot 3.3.0 documentation," n.d.). I ran into some issues because some of the

projections no longer worked such as “merc” or Mercator projection, which instead outputted a thin strip of a projection instead of the whole map. Following this, I encountered a problem with the “rotated” and “ortho” projections having an UnboundLocalError with xpt, so I wrote up a bug report to upload to GitHub (“Issues · NCAS-CMS/cf-plot,” n.d.). This helped me to get a better understanding of what is required to fix a bug in code, such as what environment is being used because varying versions of python and cf-python can cause different issues (Figure 1).

Figure 1 - My bug report on GitHub for the rotated and ortho error



moonbooks73 commented last week • edited

As we've observed I can do a default projection (cyl) with the following code, but when introducing mapset with proj as "rotated" an error occurs. I expected a rotated projection to appear with contour plots and a title. The same issue also occurs withe the "ortho" projection/

```
>>> f=cf.read('cfplot_data/ggap.nc')[0]
>>> cfp.gopen()
>>> cfp.con(f.subspace(pressure=850), lines=False, title = "rotated", colorbar_position=[0.1, 0.1, 0.8, 0.02], colorbar_c
>>> cfp.gclose()
>>> cfp.gopen()
>>> cfp.mapset(proj="rotated")
>>> cfp.con(f.subspace(pressure=850), lines=False, title = "rotated", colorbar_position=[0.1, 0.1, 0.8, 0.02], colorbar_c
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/natalia/anaconda3/lib/python3.12/site-packages/cfplot/cfplot.py", line 1097, in con
    map_title(title)
  File "/home/natalia/anaconda3/lib/python3.12/site-packages/cfplot/cfplot.py", line 9014, in map_title
    plotvars.mymap.text(xpt, ypt, title, va='bottom',
                        ^^^
UnboundLocalError: cannot access local variable 'xpt' where it is not associated with a value
```

Environment

Using the `main` branch:

```
Platform: Linux-5.4.0-131-generic-x86_64-with-glibc2.31
HDF5 library: 1.12.1
netcdf library: 4.8.1
udunits2 library: /home/natalia/anaconda3/lib/libudunits2.so.0
esmpy/ESMF: not available
Python: 3.12.2
dask: 2024.5.0
netCDF4: 1.6.2
psutil: 5.9.0
packaging: 23.2
numpy: 1.26.4
scipy: 1.13.1
matplotlib: 3.8.4
cftime: 1.6.4
cfunits: 3.3.7
cfplot: 3.3.0
cfdm: 1.11.1.0
cf: 3.16.1
```

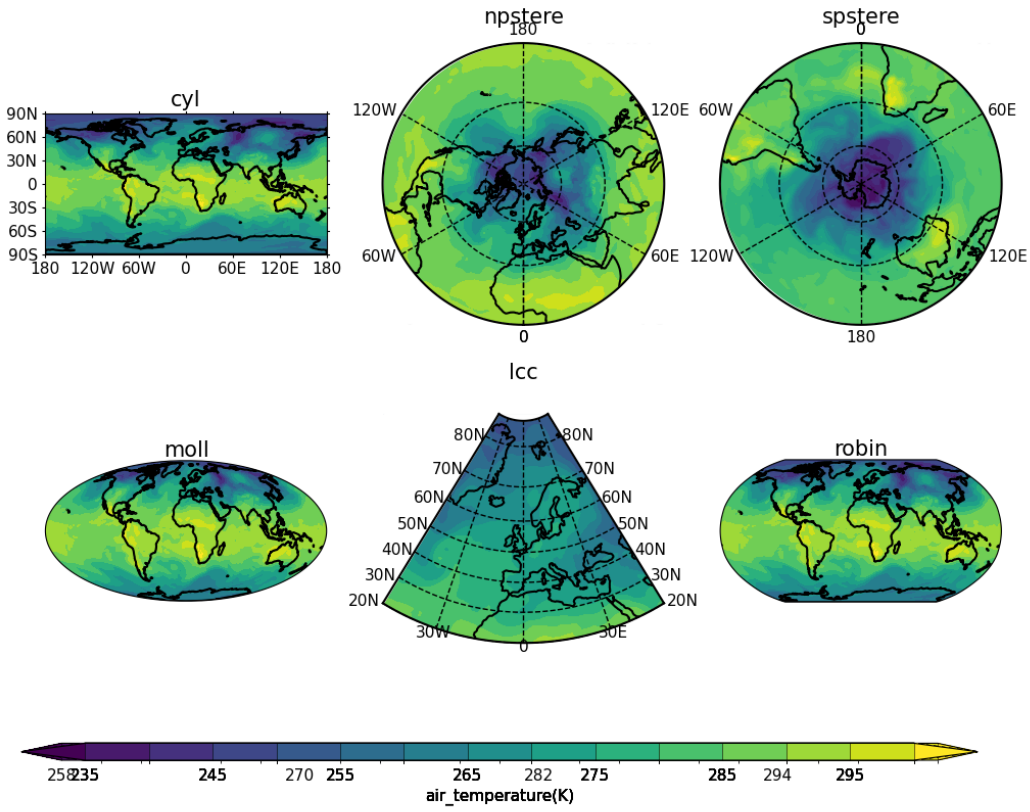
1

See Figure 18 - Code for GitHub ortho and rotated bug in the Appendix for the full code.

In order to write my recipe, I tested each of the projections, I had to read the field in from a file I used data from cfplot_data, with ggap.nc, but tas_A1.nc can also be used. In the end I used the Cylindrical (Default), North Pole Stereographic, South Pole Stereographic, Mollweide, Cropped Lambert Conformal and Robinson projections.

Throughout my programming journey, I came across various errors such as overlapping colour bars at the bottom, as each projection would add their own, making it confusing and illogical (see Figure 2). You can see the code for this in the appendix, Figure 19 - Code for error with overlapping .

Figure 2 - Subplots with multiple projections with colour bar error



To fix this I added an else statement so only the last one added a colour bar.

See Results and Discussion Section for the final code.

Second Recipe – Subplots with different Colour Maps

For my second recipe I wanted to display the various colour maps/scales (this is the colouring of the plot based on the relative values of the data, displayed in the bar below plots) available to make it clearer what ones are most suitable. For instance, the rainbow/jet colour scale is no longer used (Crameri et al., 2021) because it is not perceptually uniform, this means equal changes in data values do not correspond to equal changes in colour due to its uneven gradient; a change from green to yellow may not be perceived equally or as dramatically as a change from orange to yellow. This can

effectively lead to data manipulation, creating unintended biases which can have consequences on the wider public and scientific community as data is misinterpreted due to artificial boundaries and patterns (Crameri et al., 2020).

Some colour maps may also cause issues for individuals with colour-vision deficiencies who may be unable to perceive some of the colour changes, making the plots and therefore data inaccessible.

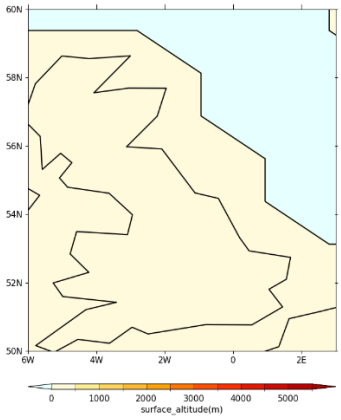
Therefore, I wanted to write a recipe to display all the colour scales available so scientists could choose the ones best suited. I used cf-plot documentation to find the ones available ("Colour scales — cf-plot 3.3.0 documentation," n.d.). There were three main sets the perceptually uniform set (viridis etc) for data with no zero data, scales enhanced for colour blindness and orography (land shape) and bathymetry (shape of water beds). See the Results and Discussion Section for my final code, its outputs and recipe.

Third Recipe – Comparing two datasets: Snow against elevation

My third and final recipe was to plot the correlation between elevation and snow cover and find the value relating to this. This recipe could therefore be used in a similar way to find the correlation between any two datasets such as population of a species and elevation, to detect a correlation for where they tend to inhabit, using contour plots.

Firstly I started using some orography data to see what could be done (and making use of an orography colour scale), it gave a good view of the elevation over the globe (Figure 20) However, I realised the data was too low resolution to use, it had one data point for the entire UK, as can be seen in Figure 3.

Figure 3 - Orography for low resolution dataset.

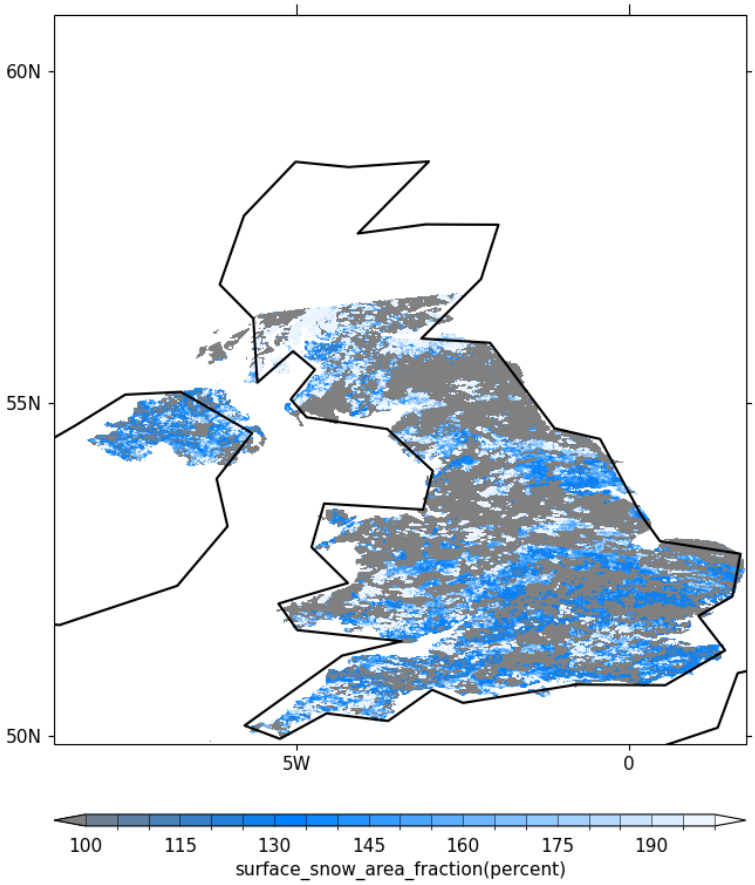


Therefore, I needed to use higher resolution data, so I used a 1 km orography dataset ("data: NOAA NGDC GLOBE topo," n.d.) I experimented with stippling (Appendix, Figure 21), vertical line plots (Appendix, Figure 23) and other methods on this dataset (Code can be found in Appendix, Figure 22) . When doing line plots I also had the option to use

a mean across the area (Appendix, Figure 24) , use data from different days as well as ensure the axis were calibrated properly (Figure 25). The code for this can be seen in the Appendix, Figure 26.

Next, I needed snow cover data, and I accessed some for over the UK over a period of seven days ("Snow Cover Extent 2017-present (raster 500 m), Europe, daily – version 1," n.d.), this meant I could look at the different data for each day which gave me some interesting plots as I looked through. However, the data seemed to present fraction of snow cover as over 100% (see Appendix, Figure 27) so I had to do some normalisation in my data in order for it to run. I also needed to find a suitable colour scale, however most seemed to make it look like elevation, so I made my own ranging from white (complete snow) to blue and then grey for no data.

Figure 4 - Snow cover with my colour scale.



See Results and Discussion for the final plots and Recipe.

Results and Discussion

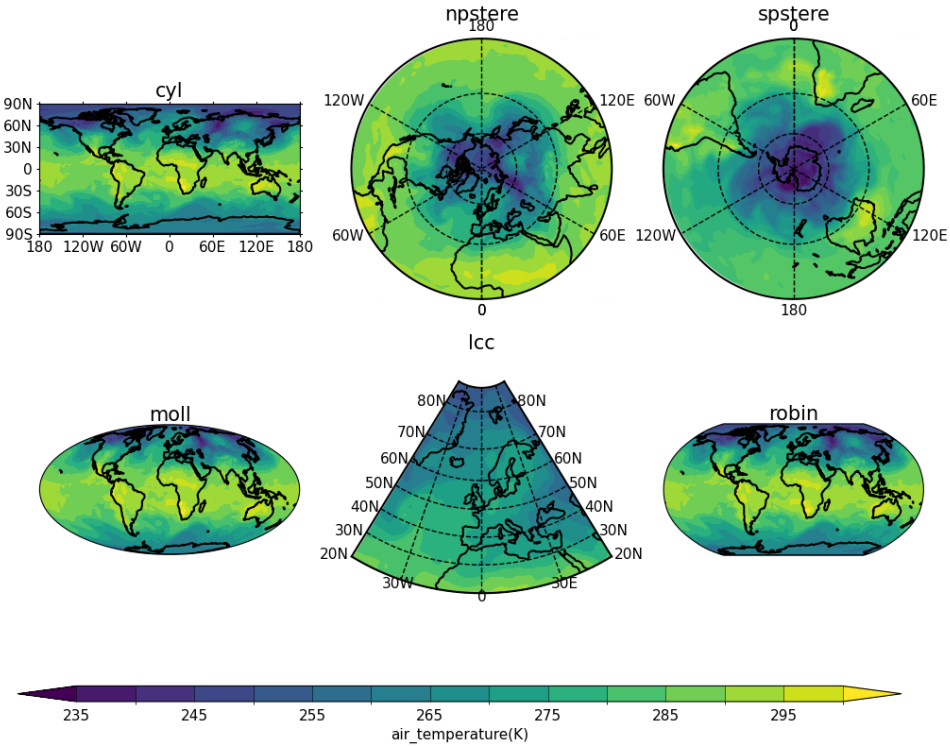
First Recipe – Subplots with different projections

The final code for the different projections is in Figure 5, the outputted plots in Figure 6.

Figure 5 - Final Code for Subplots of different projections.

```
1. import cfplot as cfp
 2. import cf
 3. f=cf.read("~/cfplot_data/ggap.nc")[0]       #Reading in the file, you can also use
tas_A1.nc, and if so instead of pressure=850 use time=15
 4. cfp.gopen(rows=2, columns=3, bottom=0.2, file="final_projections.png") #create the plot
file, we are doing 6 projections and so need it to be 2x3
 5. projtypes = ["cyl", "npstere", "spstere", "moll", "lcc", "robin"] #you could use different
projections, here it is Cylindrical/Default, North Pole Stereographic, South Pole Stereographic,
Mollweide, Cropped Lambert Conformal and Robinson
 6. for i, proj in enumerate(projtypes):
 7.     cfp.gpos(i+1) #gpos takes 1 as first value, otherwise, there are errors
 8.     if projtypes[i] == "lcc": #lcc is the only one that needs to be over a certain area,
Europe
 9.         cfp.mapset(proj='lcc', lonmin=-50, lonmax=50, latmin=20, latmax=85)
10.     else:
11.         cfp.mapset(proj=projtypes[i])
12.     if i ==len(projtypes)-1:
13.          cfp.con(f.subspace(pressure=850), lines=False, title = projtypes[i],
colorbar_position=[0.1, 0.1, 0.8, 0.02], colorbar_orientation='horizontal') #to see the marking
lines need to be True, but this can be hard to read so if not needed use False
14.     else:
15.         cfp.con(f.subspace(pressure=850), lines = False, title = projtypes[i], colorbar =
False)
16. cfp.gclose()
```

Figure 6 - Final projections outputted by cf-plot.



Here is the recipe (Figure 7) formatted to be consistent with other raw-format recipes which are then processed by the documentation builder to be rendered like a Jupyter Notebook in the documentation.

Figure 7 - Recipe of Subplots with different projections.

```
"""
Plotting Contour Subplots with different Projections
============================================
In this recipe, we will plot different projections for the same data to illustrate
visually the ones available in order to decide more visually which is suitable.
"""

# %%
# 1. Import cf-python and cf-plot:

import cfplot as cfp

import cf

#%%
# 2. Read the field in:
# Here I've used sample data ggap.nc (and later pressure=850),
# but you could use tas_A1.nc (with time=15)

f=cf.read("~/cfplot_data/ggap.nc")[0]

#%%
# 3. Create the file with subplots:
# If you are changing the number of subplots ensure the number of rows * number
# of columns = the number of subplots/projections
# Here we are doing 6 projections so 2x3 is fine

cfp.gopen(rows=2, columns=3, bottom=0.2, file="projections.png")

#%%
# 4. List the projection types being used:
# Here we are using Cylindrical/Default, North Pole Stereographic,
# South Pole Stereographic, Mollweide, Cropped Lambert Conformal and Robinson
# However you could also use other such as "rotated", "ortho" or
# "merc", "ukcp", "osgb", or "EuroPP"
# https://ncas-cms.github.io/cf-plot/build/user_guide.html#appendixc

projtypes = ["cyl", "npstere", "spstere", "moll", "lcc", "robin"]

#%%
# 5. We then use a for loop to cycle through all the different projection types:
# Only gpos has 1 added because it can only take 1 as its first value, otherwise there are
# errors. There are if statements for some projections (lcc, OSGB and EuroPP) as they have
# specific requirements for their contour.
# However, OSGB and EuroPP will require quite different data anyway.

for i, proj in enumerate(projtypes):
    cfp.gpos(i+1)
    if projtypes[i] == "lcc":
        cfp.mapset(proj='lcc', lonmin=-50, lonmax=50, latmin=20, latmax=85)
    if (projtypes[i]== "OSGB") or (projtypes[i] =="EuroPP"):
        cfp.mapset(proj=projtypes[i], resolution='50m')
    else:
        cfp.mapset(proj=projtypes[i])
    if i ==len(projtypes)-1:
         cfp.con(f.subspace(pressure=850), lines=False, title = projtypes[i],
colorbar_position=[0.1, 0.1, 0.8, 0.02], colorbar_orientation='horizontal') #to see the marking
lines need to be True, but this can be hard to read so if not needed use False
    else:
        cfp.con(f.subspace(pressure=850), lines = False, title = projtypes[i], colorbar =
False)
```

```
59. cfp.gclose()
```

This will therefore make it easier for Earth Scientists to visualise and see which plot could be the best fit to better represent and analyse their data.

Second Recipe – Subplots with different Colour Maps

The code in Figure 8 shows how the different colour maps/scales can be accessed, making use of enumerate to cycle through the assorted colour scales. Lines 4 and 5 can be replaced with the code from Figure 9 to display the scales enhanced for colour blindness and in Figure 10 for orography and bathymetry.

Figure 8 - Code for multiple plots with perceptually uniform colour scales (no zero value)

```
 1. import cfplot as cfp
 2. import cf
 3. f=cf.read('~/cfplot_data/ggap.nc')[0]
 4. colour_scale = ["viridis", "magma", "inferno", "plasma", "parula", "gray"]
 5. cfp.gopen(rows=2, columns=3, bottom=0.2, file="manyColourPlot.png")
 6. for i, colour_scale in enumerate(colour_scale):
 7.     cfp.gpos(i+1)
 8.     cfp.mapset(proj="cyl")
 9.     cfp.cscale(colour_scale[i])
10.     if i == len(colour_scale)+1:
11.         cfp.con(f.subspace(pressure=850), lines=False, title = colour_scale[i],
colorbar_position=[0.1, 0.1, 0.8, 0.02], colorbar_orientation='horizontal')
12.     else:
13.         cfp.con(f.subspace(pressure=850), title = colour_scale[i], lines=False)
14. cfp.gclose(view=True)
```

Figure 9 - Code for plots with NCAR command language colour scale enhanced for colour blindness

```
4. colour_scale =
["StepSeq25","posneg_2","posneg_1","BlueDarkOrange18","BlueDarkRed18","GreenMagenta16","BlueGree
n14","BrownBlue12","Cat12"]
5. cfp.gopen(rows=3, columns=3, bottom=0.1)
```

Figure 10 - Code for plots with Orography/bathymetry colour scales.

```
4. colour_scale = ["os250kmetres", "wiki_1_0_2", "wiki_1_0_3", "wiki_2_0", "wiki_2_0_reduced",
"arctic"]
```

Figure 11 - Colour plots for perceptually uniform colour scales.

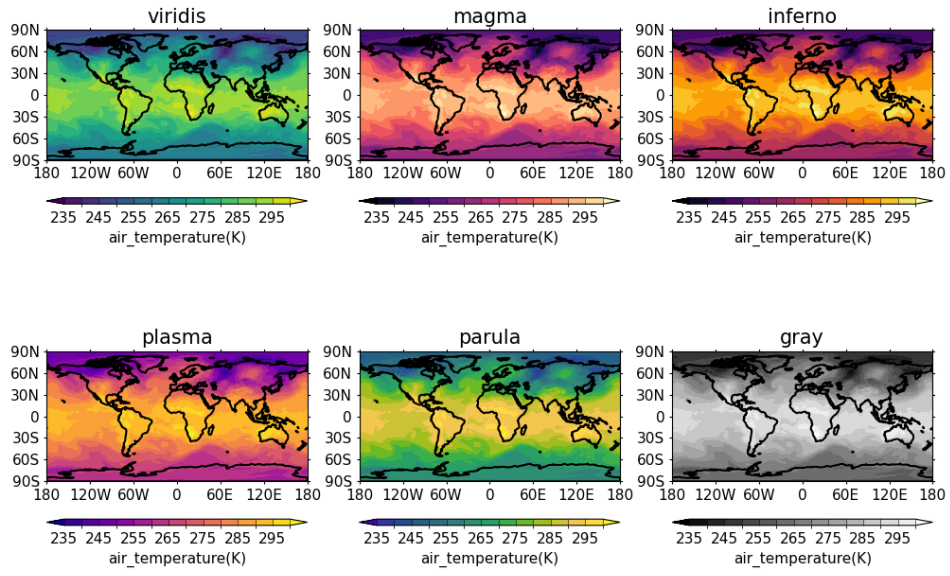


Figure 12 - Colour plots for NCAR command language colour scale enhanced for colour blindness

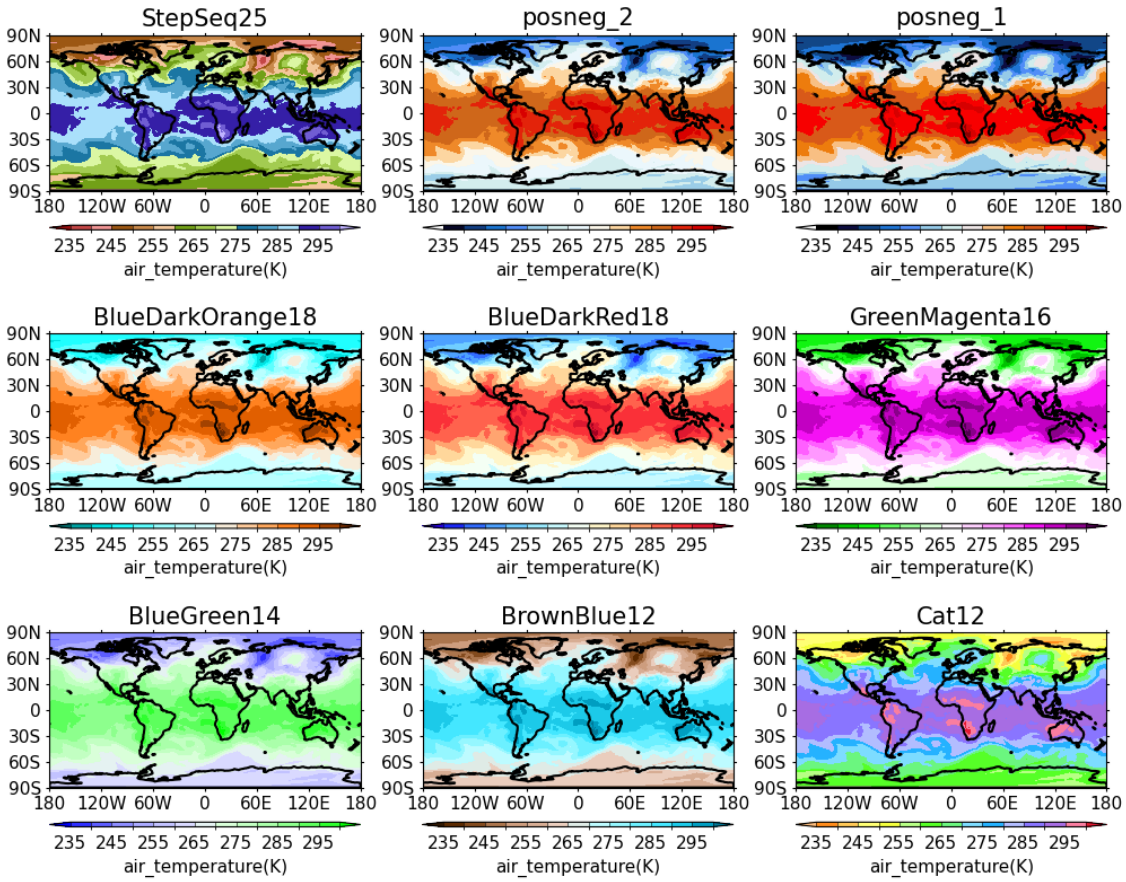


Figure 13 - Colour plots for Orography/bathymetry colour scales

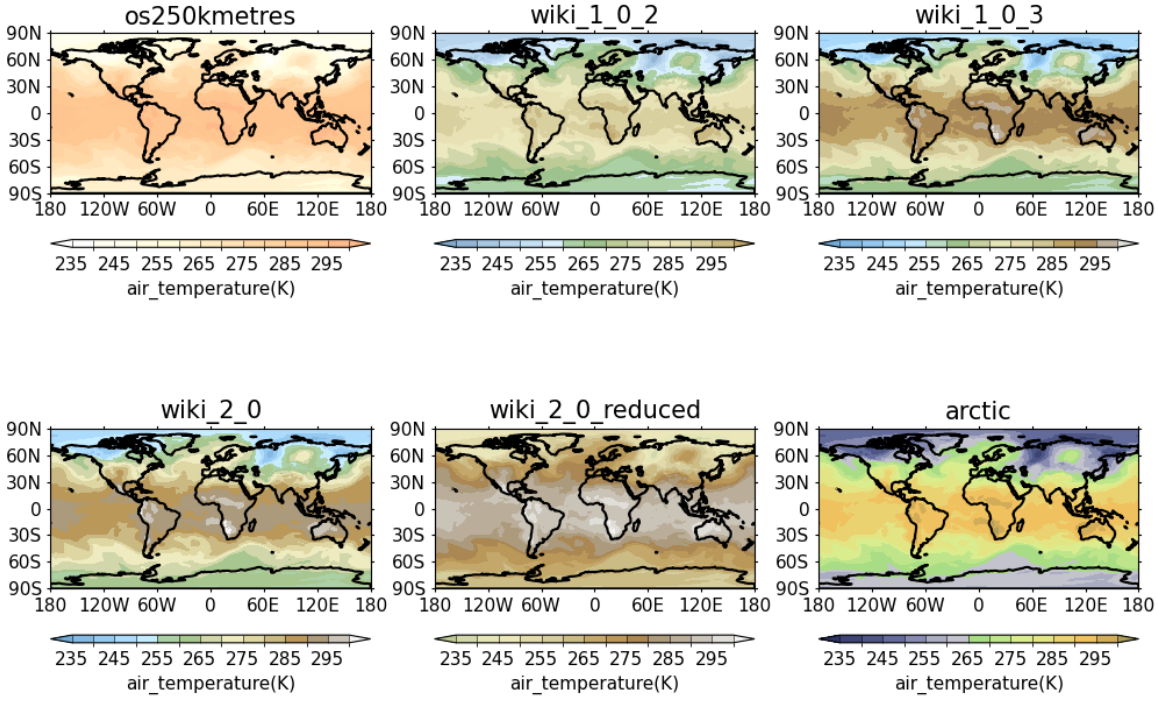


Figure 14 - Final recipe for Subplots with different colour scales

```
"""
Plotting Contour Subplots with different Colour Maps/Scales
===========================================
In this recipe, we will plot data with different colour maps to illustrate
the importance of choosing the correct one for a plot. This is to ensure
the use of perceptually uniform scales and avoid unintended bias.
"""

# %%
# 1. Import cf-python and cf-plot:

import cfplot as cfp

import cf

#%%
# 2. Read the field in:
# Here I've used sample data ggap.nc (and later pressure=850), but you could use tas_A1.nc
# (with time=15)

f=cf.read('~/cfplot_data/ggap.nc')[0]

#%%
# 3. Choose a set of predefined colour scales to view (based on NCAR)
# Choose a set of predefined colour scales to view (based on NCAR)
# You could also choose your own from
# https://ncas-cms.github.io/cf-plot/build/colour_scales.html
# Simply change the name in quotes and ensure the number of rows * number of columns =
# number of colour scales

#%%
# a. Perceptually uniform colour scales, with no zero value

colour_scale = ["viridis", "magma","inferno", "plasma", "parula", "gray"]
cfp.gopen(rows=2, columns=3, bottom=0.2)

#%%
# b. NCAR Command Language - Enhanced to help with colour blindness

colour_scale = ["StepSeq25","posneg_2","posneg_1","BlueDarkOrange18","BlueDarkRed18",
"GreenMagenta16","BlueGreen14","BrownBlue12","Cat12"]
cfp.gopen(rows=3, columns=3, bottom=0.1)

#%%
# c. Orography/bathymetry colour scales

# These are used to show the shape/contour of landmasses, bear in mind the example data
# we use is with pressure so doesn't accurately represent this.
# You could instead use cfp.cscale('wiki_2_0', ncols=16, below=2, above=14) or any other
# orography colour scale in a similar way

colour_scale = ["os250kmetres", "wiki_1_0_2", "wiki_1_0_3", "wiki_2_0", "wiki_2_0_reduced",
"arctic"]
cfp.gopen(rows=2, columns=3, bottom=0.2, file="ColourPlot.png")

#%%
# 5. We then use a for loop to cycle through all the different colour maps:
# Only gpos has 1 added because it can only take 1 as its first value, otherwise there are
# errors.
for i, colour_scale in enumerate(colour_scale):
    cfp.gpos(i+1)
    cfp.mapset(proj="cyl")
    cfp.cscale(colour_scale[i])
    if i == len(colour_scale)+1:
        cfp.con(f.subspace(pressure=850), lines=False, title = colour_scale[i],
colorbar_position=[0.1, 0.1, 0.8, 0.02], colorbar_orientation='horizontal')
    else:
```

```
67.         cfp.con(f.subspace(pressure=850), title = colour_scale[i], lines=False)
68. cfp.gclose(view=True)
```

Third Recipe – Comparing two plots: Snow against elevation

Firstly, I produced a contour plot for Snow Cover (Figure 15), this is now normalised, ranging from 0 to 100%, with appropriately labelled axis (I had to increase the number of labels to be more accurate). Using a colour scale I designed, I made the white areas clearly show where there is more snow.

Secondly, I produced an orography contour plot for the land elevation dataset, using the "wiki_2_0_reduced" colour scale (my earlier colour scale recipe helped me decide on this).

Figure 15 - Snow Cover Contour Plot

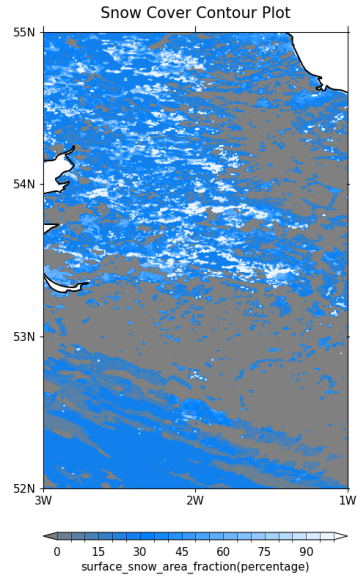
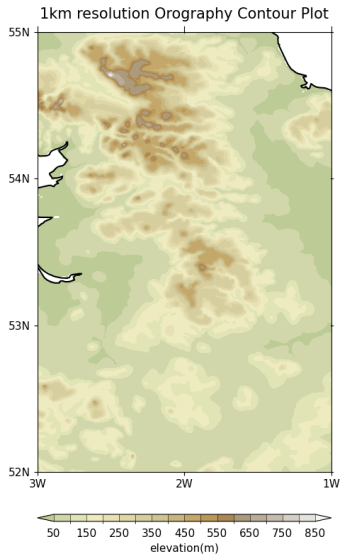


Figure 16 - 1km resolution orography contour plot



You can clearly see the correlation between elevation and snow cover, however I wanted to prove this, so the data is regridded (this means so they are on the same grid as the two datasets have different resolutions). I then used the pearsonr function from scipy ("pearsonr — SciPy v1.14.0 Manual," n.d.), the following is outputted:

```
1. PearsonRResult(statistic=0.3586727046764034, pvalue=0.0)
```

Therefore, there is a significant but minor positive correlation between snow cover and land elevation. This is because temperature decrease with altitude slowing snowmelt and increasing the likelihood of snow instead of rain. There are also other factors such as orographic lifting enhancing precipitation on windward mountain slopes; lower atmospheric pressure at higher altitudes causing air parcels to cool more rapidly; and the greater pressure drop with altitude leading to quicker cooling and promoting snow formation.

However, when calculated from other (perhaps warmer) days, when there is less snow there is no correlation, this may be because there generally isn't any snow cover that day, so any calculations are futile. I could have improved this by focusing on an area with more snow cover such as Scotland.

Below (Figure 17) shows the formed recipe ready for the documentation builder.

Figure 17 - Recipe for Comparing 2 datasets: Snow cover and Elevation

```
"""
Plot to compare two linked datasets: Snow against Elevation
===========================================
In this recipe, we will plot a dependant variable (in this example snow cover)
against an independent variable (elevation). This will be on a contour plot
separately and together, then finding the coefficient of correlation and making an
elevation line plot.
"""

# %%
# 1. Import cf-python, cf-plot and other required packages:

import cfplot as cf
import cf
import sys
import scipy.stats.mstats as mstats
import matplotlib.pyplot as plt

#%%
# 2. Read the data in:
# We are investigating the influence of the land height on the snow cover,
#so snow cover is the dependent variable. You can use different variable
# names for easier understanding.
# We are selecting the first field in the data with [0]

orog = cf.read(f"~/cfplot_data/1km_elevation.nc")[0]
snow = cf.read(f"~/cfplot_data/snowcover")[0]

snow_day = snow[0]  # first day, Jan 1st of this year (2024)

#%%
# 3. Choose the regions:
# Ensuring both datasets cover it, and that it's not too large as then
# regridding could cause the program to crash on small computers (dont
# regrid the whole map), as well as being mainly land.
# Tool for finding rectangular area for lat and lon points
# https://www.findlatitudeandlongitude.com/l/Yorkshire%2C+England/5009414/

region_in_mid_uk = [(-3.0, -1.0), (52.0, 55.0)]  # lon then lat
use_orog = orog.subspace(
    longitude=cf.wi(*region_in_mid_uk[0]),
    latitude=cf.wi(*region_in_mid_uk[1])
)

#%%
# 4. Subspace snow to same bounding box as orography data (orog)

use_snow = snow_day.subspace(
    longitude=cf.wi(*region_in_mid_uk[0]),
    latitude=cf.wi(*region_in_mid_uk[1])
)

#%%
# 5. Normalise the data
# This normalises to between 0 and 1 so multiply by 100 to get a percentage

use_snow_normal = ((use_snow - use_snow.minimum())/ (use_snow.range()))*100
```

```
print(use_snow_normal.data.stats())

#%%
# 6. Reassign the units as they removed by cf-python after calculation.
# Only do this if you are certain the units are convertible to %
use_snow_normal.override_units("percentage", inplace=True)

#%%
# 7. Plot of Snow cover contour
# First it outputs the newly formatted data, you can change the file names
# the plots will save as.
# colour_scale.txt is a colour scale specifically made and saved to show
# the data but you could make your own or use existing ones from
# https://ncas-cms.github.io/cf-plot/build/colour_scales.html
# You will also need to adjust the labels and axis for your region
print("SNOW IS", use_snow_normal)
cfp.gopen(file="snow-1.png")
cfp.cscale("~/cfplot_data/colour_scale.txt")
cfp.mapset(resolution="10m")
cfp.con(use_snow_normal,
        lines=False,
        title = "Snow Cover Contour Plot",
        xticklabels = ("3W", "2W", "1W"),
        yticklabels =("52N", "53N", "54N", "55N"),
        xticks = (-3, -2, -1),
        yticks= (52, 53, 54, 55))
cfp.gclose()
print("OROG IS", use_orog)

#%%
# 8. Plot of 1km Resolution Orography Contour
# Here the ocean doesn't get coloured out because when it is regridded it gets
# masked by the lack of data for the dependant value (snow) over the seas

cfp.gopen(file="orog-1.png")
cfp.cscale("wiki_2_0_reduced")
cfp.mapset(resolution="10m")
cfp.con(use_orog,
        lines=False,
        title = "1km resolution Orography Contour Plot",
        xticklabels = ("3W", "2W", "1W"),
        yticklabels =("52N", "53N", "54N", "55N"),
        xticks = (-3, -2, -1),
        yticks= (52, 53, 54, 55))
cfp.gclose()

#%%
# 9. Plot of Vertical Orography Line plot of 1km Resolution Elevation

lonz = use_orog.construct("longitude").data[0]
elevation_orog = use_orog.subspace(longitude = lonz)
xticks_elevation_orog = [52, 52.5, 53, 53.5, 54, 54.5, 55]

cfp.gopen(figsize=(12, 6), file ="orography_vertical.png")

cfp.lineplot(
    x=elevation_orog.coord('latitude').array,
    y=elevation_orog.array.squeeze(),
    color="black",
    title="Orography Profile over part of UK",
    ylabel="Elevation (m)",
    xlabel="Latitude",
    xticks=xticks_elevation_orog,
)

cfp.plotvars.plot.fill_between(
    elevation_orog.coord('latitude').array,
    0,
    elevation_orog.array.squeeze(),
    color='black',
```

```
    alpha=0.7
)
cfp.gclose()

#%%
# 10. Regrid the data to have a comparable array.
# Here the orography file is regridded to the snow file
# as snow is higher resolution (if this doesn't work switch them).
# The regridded data is saved as a file, when rerun this could be
# commented out and instead read in the regridded file
# as regridding takes quite awhile

reg = use_orog.regrids(use_snow_normal, method="linear")
print("REGRIDDED IS", reg, type(reg))
cf.write(reg, "regridded_data.nc")
print("DONE WRITING OUT REGRID FILE")
# This plots the regridded file data
cfp.gopen(file="regridded-1.png")
cfp.con(reg, lines=False)
cfp.gclose()

reg = cf.read("~/cfplot_data/regridded_data.nc")[0]

#%%
# 11. Compare the elevation and snow cover
# The "reg" field is the elevation data on the same grid as the snow data.
# compare this to the snow data itself "use_snow"

reg_data = reg.data
snow_data = use_snow_normal.data
print("(REGRIDDED) OROG DATA IS", reg_data, reg_data.shape)
print("SNOW DATA IS", snow_data, snow_data.shape)

#%%
# 12. Squeeze snow data to remove the size 1 axes

snow_data = snow_data.squeeze()

#%%
#  13. Final statistical calculations

sys.setrecursionlimit(1500)
coeff = mstats.pearsonr(reg_data.array, snow_data.array)
print(coeff)
```

Conclusion

Throughout my placement I made three recipes, Subplots with Different Projections, Subplots with Different Colour Scales and comparing two datasets (snow and elevation).

My first recipe, Subplots with Different Projections, will help scientists to choose the most suitable map view for data, allowing for better representation and analysis of data with clearer visualisation, allowing patterns to be spotted easier. My second recipe, Subplots with Different Colour Scales, was written to allow scientists to choose the most appropriate colour maps such as orography/bathymetry colour maps for representing land and ocean bed shapes respectively. This should also ensure the use of perceptually uniform colour scales, decreasing the likelihood of unconscious bias in data representation for the public and scientists; as well as enhanced scales for colour

blindness, increasing data accessibility. My final recipe, comparing two datasets, should make it easier for scientists to create plots and calculate correlation between dependant and independent variables. This allowed me to see the positive correlation between elevation and snow cover due to the decreased temperature and pressure.

To conclude, I hope these recipes are useful in the Earth science data analysis field to improve accessibility, ease of use and decrease unconsciously biased data.

Appendix

Figure 18 - Code for GitHub ortho and rotated bug

```
1. 1. >>> f=cf.read('cfplot_data/ggap.nc')[0]
>>> cfp.gopen()
>>> cfp.con(f.subspace(pressure=850), lines=False, title = "rotated", colorbar_position=[0.1,
0.1, 0.8, 0.02], colorbar_orientation='horizontal')
>>> cfp.gclose()
>>> cfp.gopen()
>>> cfp.mapset(proj="rotated")
>>> cfp.con(f.subspace(pressure=850), lines=False, title = "rotated", colorbar_position=[0.1,
0.1, 0.8, 0.02], colorbar_orientation='horizontal')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>   File "/home/natalia/anaconda3/lib/python3.12/site-
packages/cfplot/cfplot.py", line 1097, in con     map_title(title)
  File "/home/natalia/anaconda3/lib/python3.12/site-packages/cfplot/cfplot.py", line 9014, in
map_title     plotvars.mymap.text(xpt, ypt, title, va='bottom',
                          ^^^
UnboundLocalError: cannot access local variable 'xpt' where it is not associated with a value
```

Figure 19 - Code for error with overlapping bars.

```
1. import cfplot as cfp
 3. import cf
 4. f=cf.read("~/cfplot_data/ggap.nc")[0]       #Reading in the file
 5. cfp.gopen(rows=2, columns=3, bottom=0.2, file="final_projections.png") #create plot file
 6. projtypes = ["cyl", "npstere", "spstere", "moll", "lcc", "robin"]
 7. for i in range(1, len(projtypes)+1):    # cycling through all the projection types
 8.     cfp.gpos(i)
 9.     if projtypes[i-1] == "lcc": #only lcc needs to be over a certain area, Europe
10.         cfp.mapset(proj='lcc', lonmin=-50, lonmax=50, latmin=20, latmax=85)
11.     else:
12.         cfp.mapset(proj=projtypes[i-1])
13.     cfp.con(f.subspace(pressure=850), lines=True, title = projtypes[i-1],
colorbar_position=[0.1, 0.1, 0.8, 0.02], colorbar_orientation='horizontal')
14. #Can be hard to see with the marking lines as True so can set to False
15. cfp.gclose()
```

Figure 20 - Low resolution data orography.

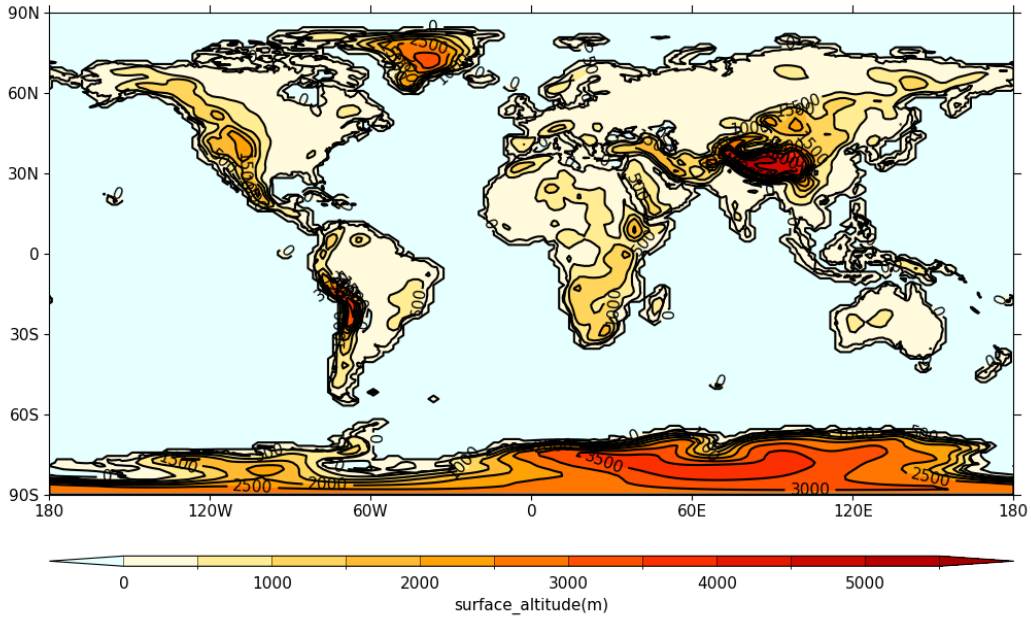


Figure 21 - Stippled 1km projection elevation.

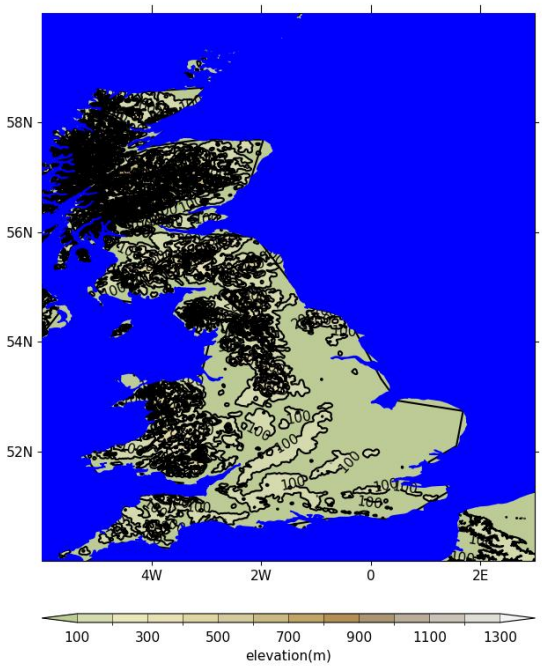


Figure 22 - Code for 1km projection contour orography.

```
1. import cfplot as cfp
 2. import cf
 3. u = cf.read("1km_elevation.nc")
 4. print(u)
 5. u = u[0]
 6. region = [(65, 95),(-0.95, 44.53) ]
 7. # # region = [(-6, 3), (50, 60)] whole uk
 8. u_subspace = u.subspace(X=cf.wi(*region[0]), Y=cf.wi(*region[1]))
 9. cfp.gopen(file="1km_projection4.png")
```

```
10. cfp.cscale('wiki_2_0_reduced')
11. cfp.setvars(ocean_color="blue")
12. cfp.con(u_subspace, lines=False)
13. cfp.gclose()
```

Figure 23 - A line plot of the elevation orography over an area

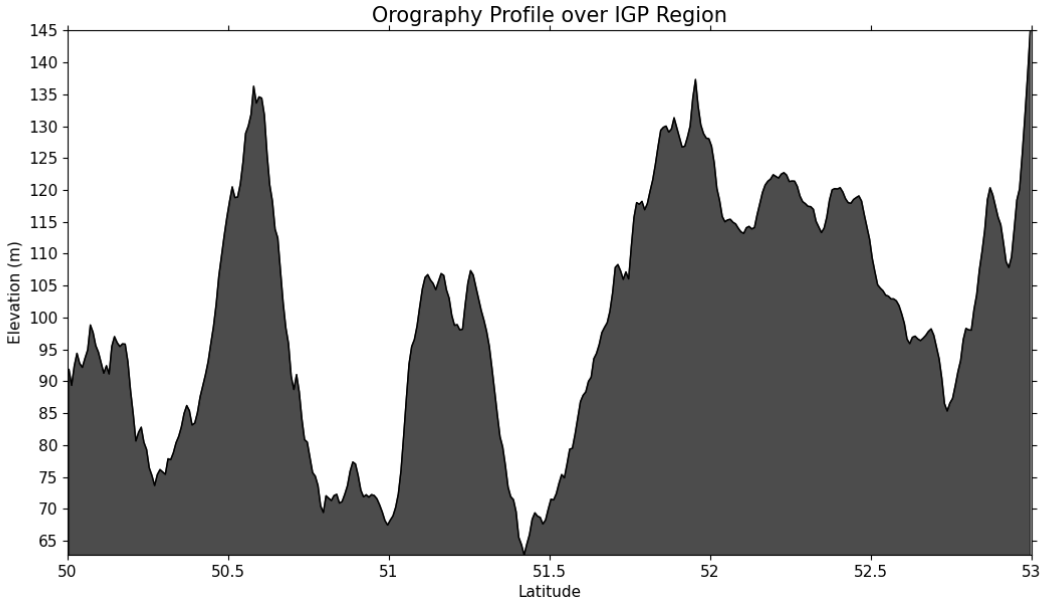


Figure 24 - Mean elevation orography line plot.

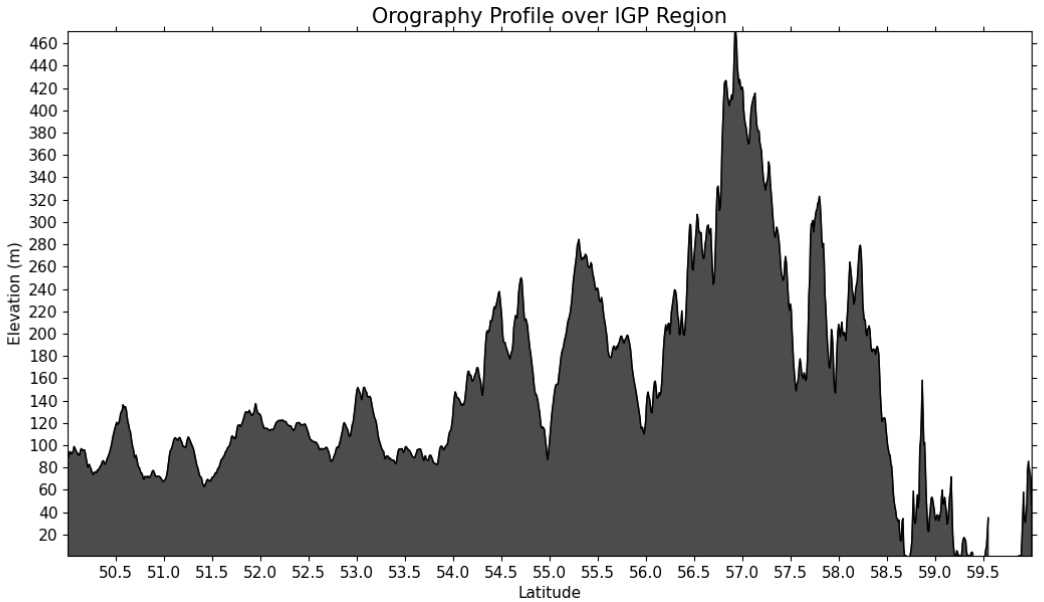


Figure 25 - Axis Calibration Issues

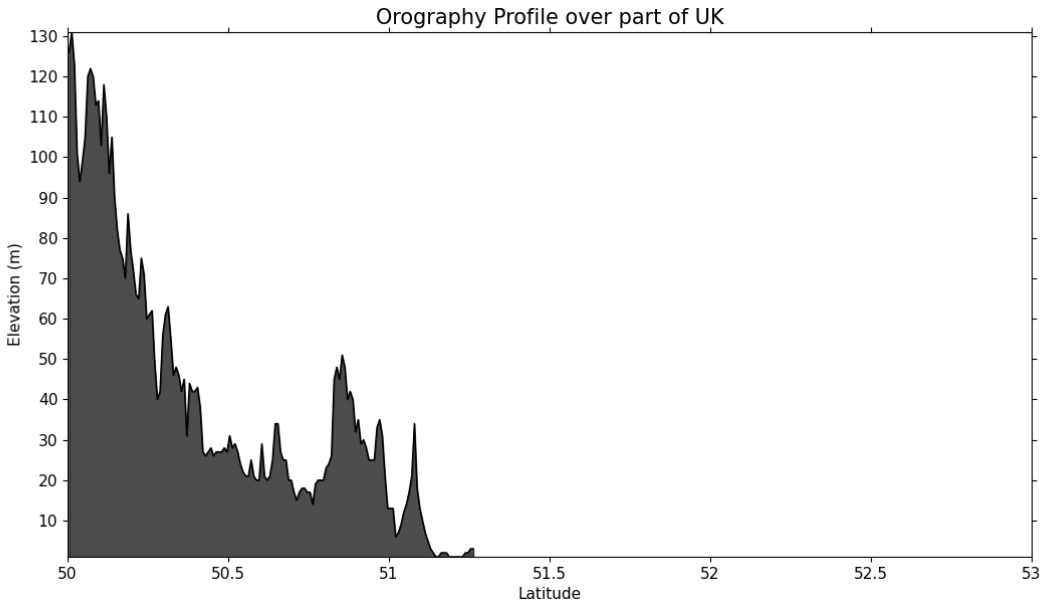
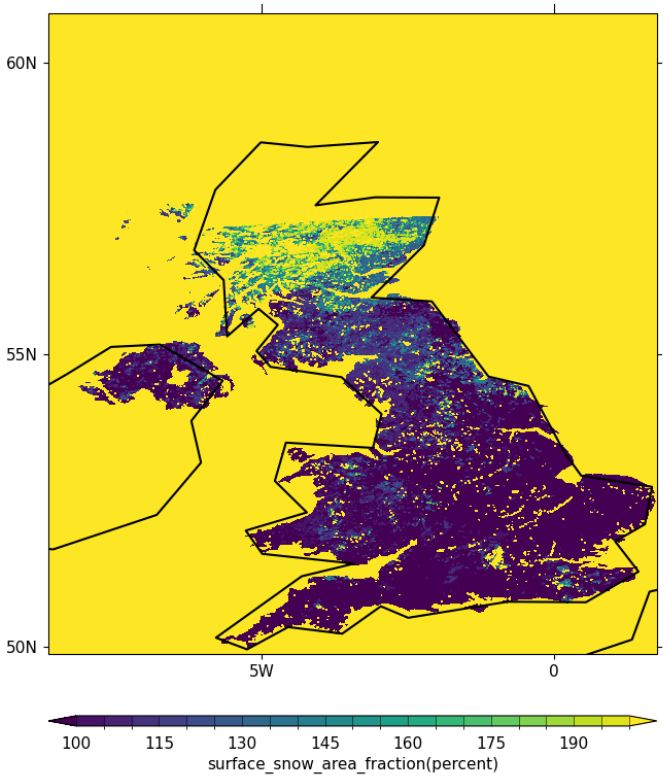


Figure 26 - Code for Line plots of 1km elevation orography.

```
1. import cfplot as cfp #imported the required libraries
 2. import cf
 3. import numpy
 4. import matplotlib.pyplot as plt
 5.
 6. orog = cf.read('~/cfplot_data/1km_elevation.nc')[0]
 7.
 8. print(orog)
 9. # uk_region = [(-6, 3), (50, 60)]
10. region = [(-6, 3), (50, 53)]       #the region of the UK I am plotting
11. orog_region = orog.subspace(longitude=cf.wi(*region[0]), latitude=cf.wi(*region[1]))
12. print(orog_region.construct("latitude").array)
13. # mean_elevation = orog_region.collapse('longitude: mean') #code for mean elevation
14.
15. elevation = orog_region.subspace(longitude = 2.9875)
16. xticks = [50, 50.5, 51, 51.5, 52, 52.5, 53]
17.
18. cfp.gopen(figsize=(12, 6), file ="orography_vertical")
19.
20. cfp.lineplot(
21.     x=elevation.coord('latitude').array,
22.     y=elevation.array.squeeze(),
23.     color="black",
24.     title="Orography Profile over part of UK",
25.     ylabel="Elevation (m)",
26.     xlabel="Latitude",
27.     xticks=xticks,
28. )     #code for the lineplot with axis labels etc
29. cfp.plotvars.plot.fill_between(
30.     elevation.coord('latitude').array,
31.     0,
32.     elevation.array.squeeze(),
33.     color='black',
34.     alpha=0.7
35. )               #filling in under the graph
36. cfp.gclose()
37. plt.show()
```

Figure 27 - UK Snow cover, Issues as over 100%.



References

A Brief Introduction to CF. (2022). Retrieved from https://www.youtube.com/watch?v=3lHgN5v8X6s

CF Conventions. (n.d.). Retrieved August 6, 2024, from https://www.youtube.com/channel/UCKLq7PCVonFJA0ec98SMFZA

CF Standard Names. (n.d.). Retrieved August 6, 2024, from https://cfconventions.org/Data/cf-standard-names/current/build/cf-standard-name-table.html

Colour scales — cf-plot 3.3.0 documentation. (n.d.). Retrieved August 6, 2024, from https://ncas-cms.github.io/cf-plot/build/colour_scales.html

Crameri, F., Shephard, G. E., & Heron, P. J. (2020). The misuse of colour in science communication. *Nature Communications*, *11*(1), 5444. https://doi.org/10.1038/s41467-020-19160-7

Crameri, F., Shephard, G., & Heron, P. (2021, October 6). How rainbow colour maps can distort data and be misleading. Retrieved August 6, 2024, from http://theconversation.com/how-rainbow-colour-maps-can-distort-data-and-be-misleading-167159

data: NOAA NGDC GLOBE topo. (n.d.). Retrieved August 7, 2024, from https://iridl.ldeo.columbia.edu/SOURCES/.NOAA/.NGDC/.GLOBE/.topo/datafiles.html

Download LMDE 6 - Linux Mint. (n.d.). Retrieved August 7, 2024, from https://www.linuxmint.com/download_lmde.php

Download/Install — cf-plot 3.3.0 documentation. (n.d.). Retrieved August 6, 2024, from https://ncas-cms.github.io/cf-plot/build/download.html

Examples — Matplotlib 3.9.1 documentation. (n.d.). Retrieved August 7, 2024, from https://matplotlib.org/stable/gallery/index.html#

Hassell, D., & Bartholomew, S. (2020). cfdm: A Python reference implementation of the CF data model. *Journal of Open Source Software*, *5*(54), 2717. https://doi.org/10.21105/joss.02717

Installing conda — conda 24.7.2.dev28 documentation. (n.d.). Retrieved August 7, 2024, from https://conda.io/projects/conda/en/latest/user-guide/install/index.html

Issues · NCAS-CMS/cf-plot. (n.d.). Retrieved August 7, 2024, from https://github.com/NCAS-CMS/cf-plot

Micromamba Installation — documentation. (n.d.). Retrieved August 8, 2024, from https://mamba.readthedocs.io/en/latest/installation/micromamba-installation.html

NCAS CMS. (n.d.). Retrieved August 6, 2024, from https://github.com/NCAS-CMS

NCAS-CMS/cf-python. (2024, August 1). Python, NCAS CMS. Retrieved from https://github.com/NCAS-CMS/cf-python (Original work published September 13, 2019)

NCAS-CMS/cf-tools-training: NCAS data tools training material. (n.d.). Retrieved August 7, 2024, from https://github.com/NCAS-CMS/cf-tools-training

pearsonr — SciPy v1.14.0 Manual. (n.d.). Retrieved August 9, 2024, from https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.mstats.pearsonr.html#scipy.stats.mstats.pearsonr

Project Jupyter. (n.d.). Retrieved August 7, 2024, from https://jupyter.org

Projections in cf-plot — cf-plot 3.3.0 documentation. (n.d.). Retrieved August 7, 2024, from https://ncas-cms.github.io/cf-plot/build/projections.html

Recipes using cf — Documentation. (n.d.). Retrieved August 6, 2024, from https://ncas-cms.github.io/cf-python/recipes/index.html

Snow Cover Extent 2017-present (raster 500 m), Europe, daily – version 1. (n.d.). Retrieved August 14, 2024, from https://land.copernicus.eu/en/products/snow/snow-cover-extent-europe-v1-0-500m

Unidata/netcdf4-python. (2024, August 5). Cython, NSF Unidata. Retrieved from https://github.com/Unidata/netcdf4-python (Original work published January 13, 2014)

Bibliography

2020 CF Workshop. (n.d.). Retrieved August 6, 2024, from https://cfconventions.org/Meetings/2020-Workshop.html

2022 CF Training Workshop. (n.d.). Retrieved August 6, 2024, from https://cfconventions.org/Training/2022-Training-Workshop.html

Built-in Functions. (n.d.). Retrieved August 6, 2024, from https://docs.python.org/3/library/functions.html

cf-plot documentation — cf-plot 3.3.0 documentation. (n.d.). Retrieved August 6, 2024, from https://ncas-cms.github.io/cf-plot/build/

cf-plot user guide — cf-plot 3.3.0 documentation. (n.d.). Retrieved August 6, 2024, from https://ncas-cms.github.io/cf-plot/build/user_guide.html

cf-tools-training/full_course at master · NCAS-CMS/cf-tools-training. (n.d.). Retrieved August 6, 2024, from https://github.com/NCAS-CMS/cf-tools-training/tree/master/full_course

Department of Meteorology - University of Reading. (n.d.). Retrieved August 9, 2024, from https://www.reading.ac.uk/meteorology/

Download/Install — cf-plot 3.3.0 documentation. (n.d.). Retrieved August 6, 2024, from https://ncas-cms.github.io/cf-plot/build/download.html

Hassell, D., & Bartholomew, S. (2020). cfdm: A Python reference implementation of the CF data model. *Journal of Open Source Software*, *5*(54), 2717. https://doi.org/10.21105/joss.02717

Acknowledgements

A special thanks to Sadie Bartholomew, my supervisor for her guidance throughout my placement at the University of Reading, helping me to learn and understand cf python and for all her assistance and guidance in writing these recipes. Many thanks to the University of Reading, NCAS and the Department of Metrology for warmly hosting and supporting me during my placement.